

2026 年春季 操作系统期末考试（大班）

注：本试卷为回忆卷，以供选修大班 OS 的同学复习。注意，2026 年期末春季期末考试的风格与以往有较大的差别。

一、系统调用简答题（对应 Lab2，20 分）

题干讲到了 Ring0 和 Ring3、syscall 等内容。

1. 为什么要用 `int 0x80` 来调用操作系统内核函数，而不能直接在用户态调用内核函数？
2. `irqHandle` 和 `syscallHandle` 的区别？如何添加 `open` 系统调用
(此处补充了 `syswrite` 函数的代码实现)
3. 为什么内核不能直接访问 `userspace` 的 `str`？(`str` 为 `syswrite` 的一个参数，题目说的意思是，为什么要转换段基址)；体现了操作系统什么样的安全思想？
4. 上述 `syswrite` 函数的 bug（没有检查权限，可以读取内核）

二、进程调度（对应 Lab3）

1. `fork` 和 `vfork` 对 `parent` 内存采取什么样的处理？`vfork` 为什么只能最多有个未执行的 `exec`？
2. (给出了一段 `schedule` 的代码) 问实验中实现的是什么调度策略？
3. (给了时钟中断的代码) 问除了时钟中断，`schedule()` 还会在什么时候被调用。如果时钟中断，中不调用 `schedule()`，会出现什么情况？其他进程能否被调度？
4. `schedule` 中没有实现上下文切换，请问在实验中，具体是那段代码实现的呢？`schedule` 和上下文切换分开有什么好处？

三、Fat12 文件系统（Lab5）

题干：在实验中，我们实现了 `Fat12xxx`；

1. 一个文件 20kb，一个簇 512B，需要多少个簇？在 `Fat12` 中，怎么找 `next cluster`
2. 比较 `Fat12` 和 `Ext` 的随机读写性能。
3. (给出了一小段代码，内容是在读取到 `cluster` 边界时的处理：读到 `cluster` 的末尾，然后找下一个 `cluster`) 问如果 `fork` 出的子进程和父进程同时读，会出现什么问题？怎么解决
4. (给了两行代码，分别是 `Fatopen` 和 `fat_read`，期中 `fat_open` 的时候把 `fat_size` 给保存在了 `FileDescriptor` 里面) 问，如果往文件的末尾追加了一些内容，使用原本的 `fd` 读，能否读到新的内容？要怎么做才能读到新内容？会有什么代价？

四、磁盘

(题干里面已经介绍了 `Aging`、`FIFO`、`LRU` 等算法)

1. `R/W` 位的作用
2. `Aging` 算法 demo (给了几个页表项，要求写出下一次更新的结果，淘汰什么页面)
3. 如果 `CLOCK` 算法中，所有位的 `R` 都是 1，会发生什么；`CLOCK` 算法和 `FIFO` 算法的异同；`CLOCK` 和 `Aging` 算法相比的优缺点。
4. 一个进程循环读写 `0,1,2,...,9` 这几个页面，分配了 4 个物理页框，问 `FIFO/CLOCK/AGING` 的缺页率；为了让 `CLOCK` 和 `AGING` 发挥作用，需要进程满足什么条件？

五、IO

1. 预读和延迟写的原理和作用
2. 若有大量的随机写，对预读有什么影响？预读的性能会不会比不预读还低？

3. 读必须要阻塞，写可以立即返回，请你设计一个 IO 调度策略，差异化处理读写。

六、PV 操作

BufferA 有 M 个空位，BufferB 有 N 个空位，Producer 采集视频，把处理好的 Frame 放到 BufferA；Consumer 从 BufferA 中取 Frame，进行目标检测，结果放到 BufferB。题干中给出了 Producer 和 Consumer 的实现。

1. 在实际中发现 BufferA 还没有满，但是 Producer 和 Consumer 都已经阻塞了，为什么？
2. 改进 1 中的问题，并且修改代码以提高效率。

—

补充：

（背景：大班做了 5 个 xuf lab，每一个 Lab 都要求在实验报告里面回答一些问题，这些考察的知识点和考试的内容有不少的重合相似之处；读者应该参考自己本学期 Lab 的内容进行针对性复习）

以下内容为本学期，涉及到考试的相关 Lab 内容如下：

Lab2

实验报告要求

报告中要求回答以下问题。

- ①为什么用户程序不能直接调用内核函数，而要通过 `int 0x80` 这种特殊的方式？
- ②当用户程序执行 `int 0x80` 触发系统调用时，CPU 自动将哪些内容压入栈中？内核处理完成后通过 `iret` 返回，返回时栈帧是如何变化的？
- ③在 `syscallWrite` 函数中，为什么需要通过 `getSegBase(tf->ds)` 获取用户段的基址，将用户态的虚拟地址转换为物理地址后才能访问？
- ④app被kernel加载到内存后，为什么不能直接跳转执行？框架代码此处是如何处理的，请详细描述。

Lab3

实验报告要求

报告中要求回答以下问题。

- `fork`与`vfork`的本质区别是什么？在内存管理上有何不同？
- `vfork`为何需要限制其创建的子进程数量？它如何通过与`exec`的配合来控制子进程数量？
- 描述`schedule()`函数中上下文切换的过程。

Lab5

实验报告

请在实验报告中回答以下问题，并将答案写入 `[学号].pdf` 中：

1. Lab5 的构建流程中，`Makefile` 是如何使用 `fat_tool` 生成 `os.img` 的？`fat_tool` 主要完成了哪些工作？
2. `fat_tool create os.img` 是如何把一个普通文件初始化成 FAT12 磁盘镜像的？请结合代码说明它写入了哪些关键元数据。
3. 以 `fat_tool import os.img test.txt /data/test.txt` 为例，导入一个文件时，`fat_tool` 需要完成哪些步骤，才能让这个文件之后被找到并读出？
4. `fat_tool` 是如何在 FAT 表中分配和连接 cluster 的？请说明 FAT12 中 12-bit 表项的读写方式。
5. `fat_tool` 创建的目录项中，文件名、属性、起始簇号、文件大小这些字段分别有什么作用？它们后续如何支持 bootloader 或 kernel 读取文件？